

Сборка файла и разрешение коллизий — инженерный разбор

Тот же материал, что в академической монографии [AssemblyGuide.Academic.ru.md](#), но без формализмов ради формализмов: короче записи, больше «почему сделано именно так» и «как это ведёт себя на реальном потоке». Нумерация разделов в обеих версиях совпадает, так что ссылки взаимозаменяемы. Контекст (формат пакета, хеши H5/D3, накопление) — в [AlgorithmGuide.ru.md](#). Самый подробный вводный разбор «с нуля» — [AssemblyGuide.Tutorial.ru.md](#).

1. Что мы вообще собираем

После сканирования у декодера на руках:

- заголовок файла: **N** data-томов, **M** ECC-томов, размер файла, SHA-256 всего файла (**H3**) и **H5** — сид хеша секторов;
- принятые 64-байтные payload, разложенные по номерам секторов. У одного номера может лежать несколько **разных** payload — это версии; у каждой счётчик — сколько раз она приходила.

Сборка — это выбрать по одному payload на каждый номер, склеить первые **N** томов, обрезать до размера файла и посчитать SHA-256. Совпал с **H3** — файл готов. Не совпал — берём другую комбинацию.

Ключевое свойство: проверка одна и финальная, и она не зависит от того, как пакеты приходили. Поэтому результат всегда либо файл, бит-в-бит равный исходнику, либо честный отказ (**null**). Состояния «почти собрали, выдадим как есть» не существует в принципе.

2. Откуда берутся две версии одного сектора

- **Обычные повторы — не коллизия.** Тот же payload пришёл ещё раз → счётчик версии вырос, ничего перебирать не нужно.
- **Случайная коллизия хеша.** Проверка сектора — 72 бита SHA-256; на практике не случается, но теоретически возможна.
- **Подделка.** H5 передаётся в потоке открыто, так что любой, кто перехватил поток, может нагенерировать «валидных» секторов с любым содержимым. Вот под этот случай всё и заточено.

Подделка не может пройти финальную проверку — для этого нужен прообраз SHA-256. Максимум вреда: отказ в сборке или потраченное на перебор время. Защита трёхслойная: счётчики (повторы усиливают правду), перебор равновероятных кандидатов, финальный хеш.

3. Где это всё лежит

Слот файла — словарь «номер сектора → список версий». Версия — payload плюс счётчик. Два правила, которые всегда выполняются:

1. **Список отсортирован по счётчику по убыванию.** Первый элемент — самый подтверждённый; именно он берётся «по умолчанию».
2. **Равные счётчики не переставляются между собой.** Внутри группы равновероятных порядок стабилен; менять его умеет только прокрутка (раздел 11), и то внутри группы.

Деталь реализации, о которой стоит знать: точка ветвления держит **живую ссылку** на список версий слота, а не копию. Прокрутка двигает реальные списки слота — так дешевле, а кто хочет снимок, зовёт `GetSectorVersions` и получает копии payload.

4. Приём копии сектора (`AddSector`)

Что происходит на каждый принятый 75-байтный пакет-сектор:

1. Проверки входа: номер в пределах `0..N+M-1`, payload ровно 64 байта. Хеш уже проверил подлинность — эти проверки просто защищают API от кривых вызовов.
2. Если номер новый — создаём список, кладём версию со счётчиком 1.
3. Иначе линейно ищем побайтово совпадающий payload:
 - нашли — счётчик `+1` (с потолком `int.MaxValue`, чтобы не переполниться) и **всплытие**: версия поднимается вверх, пока сверху лежит строго менее подтверждённая. Через равные не лезем — правило 2 из раздела 3.
 - не нашли — новая версия в **конец** списка (все существующие имеют счётчик ≥ 1 , так что сортировка не ломается).

Почему линейный поиск, а не словарь по payload: в 99,9% случаев в списке одна версия, и одно 64-байтное сравнение дешевле, чем поддерживать хеш-таблицу на каждый сектор. Память тоже экономится.

Побочный эффект всплытия — «лидер» списка всегда самый подтверждённый. На этом держится вся логика «предпочтительной выборки» ниже.

5. Общий план сборки

TryAssemble

1. selected ← по versions[0] каждого номера (самые подтверждённые)
2. points ← номера, где максимум счётчика делят несколько версий
3. таких номеров нет?
 - да → одна попытка: прямая сборка → RS // разделы 6–7
 - нет ↓
4. C ← произведение количеств равновероятных версий (с зажимом в long.Max)
5. попытка №0 (текущий выбор) с замером времени t_0
6. оценка полного перебора = $t_0 \cdot C$
7. $C \leq 100\,000$ и оценка ≤ 30 с?
 - да → полный перебор (раздел 10)
 - нет → прокрутка (раздел 11)
8. снова 1 и есть ЕСС → подбор подмножества томов (раздел 12)

Обе стратегии на каждом шаге делают **одну и ту же** попытку (раздел 6, затем 7) и отличаются только тем, как готовят следующий набор версий. Отмена и бюджет времени проверяются между попытками. Стадия 3 добивает то, что им не по зубам, — без коллизий и с независимым бюджетом (раздел 12).

6. Одна попытка: прямая сборка

Всё скучно и быстро: если все N data-томов в выборке есть — склеиваем $N \cdot 64$ байт, обрезаем до размера файла, хешируем, сравниваем. Одна попытка стоит конкатенацию плюс один SHA-256. Прямой путь покрывает случай, когда потерь нет или они закрыты повторами.

7. Одна попытка: RS-добивка

Прямая не прошла, а ЕСС настроен:

1. Строим карту «какие тома на месте» по всей линейке $N+M$.
2. Если всего на месте меньше N — восстанавливать не из чего, отказ.
3. Зовём декодер Рида–Соломона: он вернёт N data-томов — целые как есть, потерянные восстановит по ЕСС. Условие восстановления простое: потерянных data не больше, чем доступных ЕСС.
4. Результат склеиваем и проверяем финальным хешем.

Две вещи, которые легко упустить:

- **ЕСС-тома — тоже сектора и тоже могут коллидировать.** Подделанный ЕСС испортит восстановление, финальный хеш не сойдётся — значит, перебор обязан щёлкать и версии ЕСС-номеров. Так он и делает: выборка строится по всей линейке $N+M$.

- RS выполняется **в каждой попытке** перебора. Перебор меняет только спорные сектора; дыры каждый раз заново закрываются ECC.

8. Какие версии вообще участвуют в поиске

Ветвление — это номер, у которого максимум счётчика делят два и более payload. Перебор ходит **только** по этим равновероятным головам. Версии со счётчиком поменьше не пробуются ни в переборе, ни в прокрутке — в этом вызове сборки их нет как нет.

Если правда оказалась в меньшинстве (подделку задублировали чаще), текущая сборка откажет. Это осознанный размен: вероятность хеш-валидной подделки мизерная, а code path для «а вдруг меньшинство право» стоил бы экспоненты. Лечение системное: продолжать приём — повторы поднимут правду до ничьей, и следующий вызов сборки увидит точку ветвления. Ну или этот сектор вообще не нужен — RS закроет.

9. Сколько комбинаций и какой поиск выбирать

Количество комбинаций — произведение размеров голов. Считается в `long`; если произведение грозит переполнением, возвращается `long.MaxValue` (в отчётах — «> long.Max»). Зажим нужен, чтобы сравнение с лимитом не падало по арифметике.

Полный перебор разрешён при двух условиях одновременно:

1. комбинаций не больше 100 000 (жёсткий лимит);
2. оценка времени укладывается в 30 секунд.

Оценка не с потолка: нулевая попытка замеряется секундомером, её длительность умножается на число комбинаций. Попытка включает SHA-256 и, возможно, RS — цена зависит от размера файла, поэтому и мерим по факту, а не по формуле. Любое из условий не выполнено — работаем прокруткой.

10. Полный перебор: одометр

Комбинации кодируются массивом индексов с модулями — «счётчик с барабанами», как одометр в старых счётчиках: правый барабан крутится первым, дошёл до предела — сбрасывается в ноль и толкает левый. Исчерпались все — комбинации кончились.

```
попытка №0 уже сделана вызывающим кодом
for i = 1 .. C-1:
    проверить отмену и бюджет (30 с)
    прокрутить одометр на один щелчок
    подставить головы точек ветвления в выборку
    выполнить попытку (разделы 6-7); успех — вернуть файл
отказ
```

Подстановка трогает только локальную выборку, порядок версий в слоте не меняется. Гарантия простая: если правильная комбинация лежит в равновероятных головах и бюджет не кончился — перебор её найдёт.

11. Прокрутка: эвристика для тяжёлых случаев

Когда комбинаций слишком много, перебирать всё нельзя. Наблюдение из практики атак: подделку дублируют во все атакованные сектора **одинаково**, то есть смещение правды по рангу везде одинаковое. Прокрутка проверяет именно такие «синхронные» сдвиги.

Механика: после каждой неудачной попытки в каждом спорном секторе первый равновероятный вариант уезжает в конец равновероятной головы. Один шаг — один сдвиг везде.

Аналогия — несколько циклических списков разной длины, крутящихся синхронно. Картина повторяется через НОК длин; на шаге **k** выбранная комбинация — «**k** по модулю длины» в каждом списке, то есть диагональ. Отсюда всё:

- **Длины взаимно простые** → НОК равен произведению → прокрутка обходит все комбинации, просто в другом порядке, чем одометр. Эквивалентна полному перебору.
- **Есть общие делители** → часть комбинаций недостижима. Пример: три головы по 2 варианта — $C = 8$, а $\text{НОК} = 2$; прокрутка проверит только две «чистые» комбинации и ни одной смешанной. Это плата за скорость.
- **Системная подделка** (одинаковый сдвиг во всех атакованных секторах) всегда лежит на диагонали — ловится одной из первых попыток.

Число состояний зажимается лимитом 100 000, бюджет и отмена проверяются на каждом шаге. Прокрутка двигает реальные списки слота, но это безопасно: перестановка внутри равновероятной головы ничего не значит для будущих сборок — любая стартовая перестановка равноправна.

12. Тихие повреждения и подбор подмножества томов

Оба поиска из разделов 10–11 рассчитаны на то, что поломка **видна**: есть коллизия — есть что перебирать. А если том формально честный — номер цел, 72-битный хеш сектора сошёлся, — но payload побит (случайная порча прошла проверку или подделка согласована с хешем)? Версия

одна, спорных нет, RS по полной карте верит испорченному тому, и хеш файла не сходится — при нулевой зацепке.

Стадия 3 переворачивает задачу: раз нечего менять в версиях — меняем карту. Подозрительные тома помечаются стёртыми, как будто не приходили, а RS восстанавливает их по оставшимся. Исключили именно брак — восстановится истина, финальный хеш подтвердит.

Порядок попыток (планировщик `SubsetMaskPlanner` — чистые функции, детерминизм, легко тестировать):

1. **База** — все коллизионные слоты целиком: не щёлкаем их версии по одной, а стираем секторы и восстанавливаем по ЕСС. Не влезает в ЕСС-запас — план пуст, стадия честно сдаётся.
2. **Уровень 1** — по одному исключаем каждый присутствующий бесспорный том, начиная с самых подозрительных: меньше подтверждений — раньше; равные — в псевдослучайном порядке с сидом от H5 (порядок прихода пакетов на результат не влияет, повторный вызов повторяет тот же план).
3. **Уровни 2–3** — пары и тройки из 64 самых подозрительных томов (шорт-лист, настраивается).

Почему отдельный кап на E — число стёртых data-томов (по умолчанию 32): цена попытки — гауссова инверсия $\sim E^2 \cdot N$. При $E \leq 32$ попытка стоит миллисекунды даже на максимальном поле ($T = 65\,535$), при $E \geq 128$ — уже секунды. Поэтому число стёртых томов ограничивается отдельно от числа попыток (100 000) и бюджета (30 с); срабатывает первое, что наступит.

Когда помогает: побитые носители и «хеш-согласованные» подделки без коллизий — один-два-три испорченных тома при ЕСС-запасе. Когда нет: массовые повреждения, не влезające в ЕСС, — честный отказ.

13. Финальная проверка и уборка (`TrimAndVerify`)

Буфер `N · 64` обрезается до размера файла (последний том кодировался с нулевой добивкой), хешуется, сравнивается с `H3`. Совпало — файл наружу. Не совпало — буферы **затираются нулями** и возвращается отказ. Затирание — не паранойя: в неудачной попытке может лежать подделка, незачем ей болтаться в куче дольше необходимого.

Итоговое правило: сборка никогда не выдаёт непроверенный результат. Отказ — это отказ, а не «примерно то».

14. Что видно снаружи

Член	Что показывает
<code>HeaderReceptionCount</code>	сколько копий заголовка поймали
<code>ReceivedSectorCount</code>	сколько номеров закрыто хотя бы одной версией
<code>ReceivedSectorCopyCount</code>	сколько всего копий секторов (сумма счётчиков)
<code>CollisionSectorCount</code>	сколько номеров с конкурирующими версиями
<code>Coverage</code>	процент закрытых номеров от <code>N+M</code>
<code>FormatValidityMap</code>	строка-карта:  принят,  коллизия,  дыра
<code>BuildCollisionMap</code>	номер → сколько версий
<code>GetSectorVersions(n)</code>	снимок версий сектора в порядке предпочтительности

Прогресс троттлится до целых процентов и не показывает 100, пока файл реально не собран. Внутри поиска прогресс идёт по комбинациям («полный перебор: k / C »), а не по байтам.

15. Пределы и настройки

Настройка	Умолчение	Что ограничивает
<code>MaxExhaustiveCombinations</code>	100 000	потолок комбинаций для полного перебора
<code>MaxHeuristicAttempts</code>	100 000	потолок состояний прокрутки (с нулевым)
<code>TimeBudget</code>	30 с	мягкий бюджет; проверяется между попытками

Стадия 3 (раздел 12) живёт в группе `SubsetSearch`:

Настройка группы <code>SubsetSearch</code>	Умолчение	Что ограничивает
<code>MaxAttempts</code>	100 000	потолок попыток стадии
<code>TimeBudget</code>	30 с	мягкий бюджет стадии
<code>MaxErasedDataVolumes</code>	32	стёртых data-томов на одну попытку (кап E)
<code>ShortlistSize</code>	64	шорт-лист для пар и троек исключений
<code>MaxExtraExclusionLevel</code>	3	максимальный размер доп. исключений

Переполнения при подсчётах не стреляют исключениями: комбинации зажимаются в `long.MaxValue`, НОК — в лимит попыток, биномиальные коэффициенты уровней — тоже в `long.MaxValue`.

16. Разбор полётов

16.1. Чистый приём

Файл 100 байт без ЕСС: два сектора, оба приняты по разу. Спора нет → одна попытка, прямая сборка, хеш сошёлся. Готово.

16.2. Один спорный сектор, два варианта

Сектор 0: варианты **A** и **B**, оба по 3 подтверждения. $C = 2 \rightarrow$ перебор: попытка №0 берёт **A**, попытка №1 — **B**. Правая найдётся максимум со второй попытки.

16.3. Два спорных сектора: 3×2

$C = 6$, НОК = 6 — взаимно простые. Перебор и прокрутка проходят **один и тот же** набор из шести попыток, только в разном порядке (одометр строками, прокрутка диагоналями). Оба полны.

16.4. Три спорных сектора: $2 \times 2 \times 2$

$C = 8$, но пусть время не влезло → прокрутка: НОК = 2, проверяются только **A+X+P** и **B+Y+Q**. Шесть смешанных комбинаций пропущены; если правда там — честный отказ. Плата за скорость: полный проход стоил бы вчетверо дороже.

16.5. Подделка победила по подтверждениям

Сектор 5: правда **A** (2 раза), подделка **B** (5 раз). Голова списка — один **B**, спорных нет, выборка всегда берёт **B**, хеш не сходится, отказ. Перебор бессилён — **A** вне поля поиска (раздел 8). Что делать: принимать поток дальше; повторы подтянут **A** до ничьей, и следующий вызов сборки начнёт ветвление.

16.6. Тихо испорченный том

N = 64, **M = 8**. Том 7 побит, но проверку сектора прошёл, версия одна. Стадии 1–2: спорных нет, единственная попытка отдаёт RS полную карту — тот верит тому 7, хеш не сходится. Стадия 3: коллизий нет, база пуста; уровень 1 исключает тома по очереди — на маске **{7}** RS восстанавливает истину, хеш сходится. Порядка N дешёвых попыток, каждая — одна инверсия при $E = 1$.

17. Что гарантировано, а что нет

Гарантировано:

- Выдали файл — его SHA-256 равен **нЗ**, то есть это исходник бит-в-бит (в рамках стойкости SHA-256).
- Все data-тома приняты однозначно — успех с первой попытки.

3. Комбинаций $\leq 100\ 000$ и время влезло — правильный набор в равновероятных головах будет найден.
4. Головы взаимно простые — прокрутка обходит всё, эквивалент перебора.
5. Ложных результатов не бывает: либо доказанный файл, либо `null`.

Не гарантировано (эвристика):

6. При общих делителях прокрутка видит только диагонали НОК.
7. Версии-меньшинства в этом вызове не проверяются вовсе.
8. Укладка в 30 секунд — эмпирика по замеру нулевой попытки.
9. Одинокое тихое повреждение вне коллизий ловится уровнем 1 стадии 3 — пока хватает ЕСС-запаса и лимитов.
10. Пары и тройки тихих повреждений — только в пределах шорт-листа и уровня исключений.

Цифры для памяти: проверка сектора 72 бита, заголовка 192 бита, арбитр — SHA-256; перебор до 100 000 комбинаций, прокрутка до 100 000 состояний, подбор до 100 000 масок при капе $E = 32$, бюджеты по 30 с. Всё это настраивается.